
IoC Extraction & Threat Intelligence Enrichment using Python

A Cybersecurity Project Report

Report By: Fareeha Naveed

Domain: Cybersecurity | Threat Intelligence | Security Automation

Abstract

In the modern threat landscape, security teams are confronted with an overwhelming volume of alerts generated by intrusion detection systems, firewalls, endpoint protection platforms, and Security Information and Event Management (SIEM) tools. Buried within these alerts are critical pieces of forensic evidence known as Indicators of Compromise (IoCs) — artifacts such as malicious IP addresses, suspicious domains, harmful URLs, and file hashes that signal the presence of an ongoing or past security incident. Manually sifting through raw log text to identify these indicators is slow, error-prone, and does not scale with the volume of data that modern Security Operations Centers (SOCs) must process on a daily basis.

This project, titled “IoC Extraction & Threat Intelligence Enrichment using Python,” addresses this challenge by building an automated pipeline that reads raw alert log files, applies Regular Expression (Regex) pattern matching to extract IPv4 addresses, URLs, domain names, and MD5, SHA1, and SHA256 file hashes, and then enriches each discovered indicator with live reputation data from two industry-standard threat intelligence platforms: VirusTotal and AbuseIPDB. Based on the reputation scores returned by these services, the system assigns each IoC a risk classification of Low, Medium, or High, allowing an analyst to immediately prioritise which indicators warrant urgent attention.

The tool is implemented as a modular Python application, with dedicated components for configuration management, regex-based extraction, API communication, risk scoring, structured logging, and report generation. The final output is produced in two universally compatible formats — JSON and CSV — making the results easy to feed into spreadsheets, dashboards, or downstream SIEM and SOAR platforms. The project was tested against both a benign sample alert file and a malicious sample alert file to validate correct behaviour under both low-risk and high-risk conditions.

By automating a task that would otherwise consume considerable analyst time, this project demonstrates a practical, real-world application of Python programming, API integration, and threat intelligence concepts, and lays the groundwork for a fully fledged threat intelligence enrichment platform suitable for use in a production SOC environment.

Table of Contents

Abstract.....	2
Table of Contents.....	3
Chapter 1: Introduction	6
1.1 Cybersecurity	6
1.2 Indicators of Compromise (IoCs).....	6
1.3 Threat Intelligence	6
1.4 Why Automation Is Needed.....	7
1.5 Importance of VirusTotal	7
1.6 Importance of AbuseIPDB.....	7
Chapter 2: Objectives.....	8
Chapter 3: Technologies Used	9
3.1 Why These Technologies Were Chosen.....	9
Chapter 4: System Architecture.....	10
4.1 main.py — Application Entry Point.....	10
4.2 config.py — Centralized Configuration	10
4.3 ioc_extractor.py — IoC Extraction Engine	11
4.4 virustotal.py — VirusTotal API Client.....	11
4.5 abuseipdb.py — AbuseIPDB API Client	11
4.6 risk_scoring.py — Risk Calculation Engine.....	11
4.7 report_generator.py — Report Writer	11
4.8 logger.py — Logging Configuration.....	12
4.9 utils.py — Shared Helper Functions.....	12
4.10 Workflow Diagram	12
Chapter 5: Regex Patterns	13
5.1 IPv4 Address Regex	13
5.2 URL Regex	13
5.3 Domain Name Regex.....	13
5.4 MD5 Hash Regex	14
5.5 SHA1 Hash Regex	14
5.6 SHA256 Hash Regex	14
5.7 Summary Table of Regex Patterns	14
Chapter 6: Threat Intelligence	16
6.1 VirusTotal	16

6.2 AbuseIPDB	16
6.3 API Requests	16
6.4 API Responses	16
6.5 HTTP Status Codes	16
6.6 Rate Limiting	17
6.7 Timeout Handling.....	17
6.8 Network Error Handling	17
Chapter 7: Risk Scoring Logic	19
7.1 Low Risk.....	19
7.2 Medium Risk	19
7.3 High Risk.....	19
7.4 Risk Decision Table.....	19
Chapter 8: Program Execution	21
8.1 Step 1 — Loading the Alert File.....	21
8.2 Step 2 — Extracting IoCs	21
8.3 Step 3 — Checking APIs	21
8.4 Step 4 — Generating Reports	21
8.5 Step 5 — Logging	21
8.6 Step 6 — Completion	22
8.7 Execution Flow Summary.....	22
Chapter 9: Sample Outputs.....	23
9.1 Project Structure in VS Code.....	23
9.2 Terminal Output.....	23
9.3 report.csv Output.....	24
9.4 application.log Output	25
9.5 VS Code — config.py	25
9.6 VS Code — main.py.....	26
9.7 Project README.....	26
Chapter 10: Testing.....	28
10.1 Testing with the Benign Alert File	28
10.2 Testing with the Malicious Alert File.....	28
10.3 Expected vs Actual Results.....	28
Chapter 11: Advantages.....	30
Chapter 12: Limitations.....	31
Chapter 13: Future Improvements	32

Chapter 14: Conclusion	33
------------------------------	----

Chapter 1: Introduction

1.1 Cybersecurity

Cybersecurity is the practice of protecting computer systems, networks, applications, and data from unauthorized access, damage, disruption, or theft. As organizations move more of their operations online, the volume and sophistication of cyber-attacks have grown dramatically, ranging from simple phishing emails to highly coordinated ransomware campaigns and nation-state sponsored intrusions. A core part of cybersecurity practice is detection — the ability to recognize, as early as possible, that an attacker is present in or attempting to access a network. Detection relies heavily on data: logs, alerts, and network traffic that must be examined and interpreted to separate genuine threats from harmless noise.

1.2 Indicators of Compromise (IoCs)

An Indicator of Compromise, commonly abbreviated as IoC, is a piece of forensic data that suggests a system may have been breached or is being targeted by malicious activity. IoCs act like digital fingerprints left behind by attackers and malicious infrastructure. Common categories of IoCs include:

- **IPv4 Addresses:** numeric network addresses associated with malicious servers, command-and-control (C2) infrastructure, or scanning activity.
- **URLs:** web addresses used for phishing pages, malware downloads, or exfiltration endpoints.
- **Domain Names:** hostnames frequently used to mask malicious IP infrastructure or to register short-lived phishing domains.
- **File Hashes (MD5, SHA1, SHA256):** unique cryptographic fingerprints of files, used to identify known malware samples regardless of the file name used.

Once an IoC is identified, it can be cross-referenced against threat intelligence databases to determine whether it has previously been associated with malicious behaviour. This project automates exactly that process: it scans alert text for IoCs using regular expressions and then checks each one against trusted reputation services.

1.3 Threat Intelligence

Threat intelligence refers to evidence-based knowledge about existing or emerging threats that can be used to inform security decisions. Rather than treating every alert in isolation, threat intelligence allows an analyst to ask: “Has this IP, domain, URL, or file hash been seen before, and if so, what was it associated with?” Threat intelligence platforms aggregate data from antivirus engines, honeypots, sandboxes, and community-submitted reports, and expose this aggregated knowledge through Application Programming Interfaces (APIs) that automated tools can query programmatically.

1.4 Why Automation Is Needed

A typical SOC can generate thousands of alerts per day, and manually identifying IoCs within free-text log entries, then individually pasting each one into a vendor's website to check its reputation, is impractical at scale. Automation solves several persistent problems faced by analysts:

- **Speed:** regular expressions can scan thousands of lines of log text in a fraction of a second, far faster than manual review.
- **Consistency:** automated extraction applies the same pattern-matching logic every time, eliminating the risk of a tired analyst missing an indicator buried in a large log file.
- **Scalability:** a script can process an arbitrary number of alert files without additional human effort, whereas manual triage time grows linearly with alert volume.
- **Prioritization:** by automatically enriching and risk-scoring each IoC, analysts can focus their limited time on the alerts that matter most, rather than treating every alert as equally urgent.

These benefits directly motivate the design of this project: a Python-based pipeline that performs extraction, enrichment, and risk scoring without requiring manual intervention at any stage.

1.5 Importance of VirusTotal

VirusTotal is a widely used threat intelligence platform, owned by Google's Chronicle (now part of Google Cloud Security), that aggregates the results of dozens of antivirus engines and website scanners into a single reputation score. When given a file hash, URL, domain, or IP address, VirusTotal returns information about how many security vendors flag that artefact as malicious, along with contextual metadata such as the first time the artefact was observed and any known malware family associations. Because VirusTotal combines the opinions of many independent antivirus engines rather than relying on a single vendor, it provides a comparatively reliable, consensus-based signal of maliciousness, which makes it an excellent first stop for automated enrichment pipelines such as the one built in this project.

1.6 Importance of AbuseIPDB

AbuseIPDB is a community-driven IP reputation database that specializes specifically in IP addresses. Network administrators and security researchers around the world submit reports of malicious activity — such as brute-force login attempts, port scanning, spam relaying, and denial-of-service traffic — that they have observed originating from specific IP addresses. AbuseIPDB aggregates these crowd-sourced reports into an abuse confidence score for each IP, which indicates the likelihood that the address is involved in malicious activity. While VirusTotal offers broad coverage across many IoC types, AbuseIPDB offers deep, behaviour-focused intelligence specifically for IP addresses, making the combination of the two services complementary rather than redundant. This is why the project queries both platforms when enriching IPv4 indicators.

Chapter 2: Objectives

The overarching goal of this project is to build a working, end-to-end automation pipeline for Indicator of Compromise extraction and enrichment. The specific objectives pursued during development are listed below.

- To design and implement a set of accurate regular expression patterns capable of extracting IPv4 addresses, URLs, domain names, and MD5, SHA1, and SHA256 hashes from unstructured, free-text security alert logs.
- To build a modular Python codebase in which each responsibility (configuration, extraction, enrichment, scoring, logging, and reporting) is isolated into its own file, making the system easy to read, test, and extend.
- To integrate the VirusTotal public API for enrichment of IP addresses, domains, URLs, and file hashes.
- To integrate the AbuseIPDB API specifically for IP address reputation checking, supplementing VirusTotal's results.
- To implement a deterministic risk scoring algorithm that converts raw reputation data from both APIs into a simple Low / Medium / High classification that any analyst, regardless of technical background, can immediately understand.
- To generate machine-readable JSON output and spreadsheet-friendly CSV output so that results can be consumed by humans, scripts, or downstream platforms such as a SIEM.
- To implement structured application logging that records every step of execution — file loading, extraction, API calls, and report generation — to support debugging, auditing, and traceability.
- To automatically process multiple alert files in a single run, demonstrated using one benign sample alert file and one malicious sample alert file, without requiring any manual selection by the user.
- To handle real-world operational concerns gracefully, including network timeouts, malformed API responses, and API rate limiting, so the tool does not crash mid-execution.
- To document the system thoroughly, producing a professional report suitable for academic submission and for onboarding future contributors to the project.

Chapter 3: Technologies Used

This project relies on a small, focused set of technologies, each chosen for a specific purpose within the pipeline. The table below summarises every technology used, along with its role in the project.

Technology	Category	Role in the Project
Python 3.13	Programming Language	Core language used to implement the entire pipeline, chosen for its readability and rich ecosystem of security and networking libraries.
Regex (re module)	Pattern Matching	Used to detect and extract IPv4 addresses, URLs, domains, and MD5/SHA1/SHA256 hashes from raw alert text.
Requests Library	HTTP Client	Used to send HTTP GET/POST requests to the VirusTotal and AbuseIPDB REST APIs and to handle their JSON responses.
VirusTotal API	Threat Intelligence Service	Provides multi-engine reputation scoring for IPs, domains, URLs, and file hashes.
AbuseIPDB API	Threat Intelligence Service	Provides a community-sourced abuse confidence score specifically for IPv4 addresses.
JSON	Data Format	Used as a structured, machine-readable output format for the final enriched IoC report.
CSV	Data Format	Used as a flat, spreadsheet-compatible output format for the final enriched IoC report.
VS Code	Development Environment	Integrated Development Environment used to write, run, and debug the project source code.

3.1 Why These Technologies Were Chosen

Python was selected as the implementation language because of its concise syntax, native regular expression support through the built-in re module, and the maturity of its HTTP client ecosystem, of which the Requests library is the de-facto standard. These properties make Python especially well suited to security automation scripts that must parse text and talk to web APIs — exactly the two core tasks this project performs. JSON and CSV were chosen as dual output formats because JSON preserves nested, structured data ideal for programmatic consumption (for example, by a SIEM ingestion pipeline), while CSV offers immediate compatibility with spreadsheet tools such as Microsoft Excel, which remain a common reporting medium in many SOC environments.

Chapter 4: System Architecture

The project follows a modular architecture in which each Python file is responsible for exactly one concern. This separation of responsibilities, often referred to as the Single Responsibility Principle in software engineering, makes the codebase easier to understand, test, and extend. The full project folder structure is shown below.

```
IOC_Extractor/  
|-- main.py  
|-- config.py  
|-- ioc_extractor.py  
|-- virustotal.py  
|-- abuseipdb.py  
|-- risk_scoring.py  
|-- report_generator.py  
|-- logger.py  
|-- utils.py  
|  
|-- benign_alert.txt  
|-- malicious_alert.txt  
|  
|-- reports/  
|   |-- report.json  
|   |-- report.csv  
|  
|-- logs/  
|   |-- application.log  
|  
|-- requirements.txt  
|-- README.md
```

4.1 main.py — Application Entry Point

main.py is the orchestrator of the entire pipeline. It is the file the user runs from the command line (python main.py), and it is responsible for tying every other module together in the correct order: loading the list of alert files from config.py, reading each file's contents, passing the raw text to the extraction module, sending each discovered IoC to the VirusTotal and AbuseIPDB clients for enrichment, calculating a risk score for each IoC, and finally invoking the report generator to write the JSON and CSV output. main.py also prints a clean, colour-coded summary banner directly to the terminal so a user can see, at a glance, what the script discovered without needing to open the generated report files.

4.2 config.py — Centralized Configuration

config.py acts as the single source of truth for every setting the application needs: file system paths (where alert files live, where reports should be written, where the log file is stored), the base URLs of the

VirusTotal and AbuseIPDB APIs, the API keys used to authenticate with those services, and the network request timeout value. Centralising configuration in one file means that if a path or API endpoint ever needs to change, a developer only has to update it in a single location rather than hunting through every module.

4.3 ioc_extractor.py — IoC Extraction Engine

This module contains all of the regular expression patterns used to detect IoCs within raw alert text, along with the logic that applies each pattern, removes duplicate matches, and returns a clean, structured list of extracted indicators (each tagged with its IoC type, such as IPv4, URL, Domain, MD5, SHA1, or SHA256). Keeping every regex pattern in one dedicated module, rather than scattering them throughout the codebase, makes the extraction logic easy to test in isolation and easy to extend in the future with new IoC types.

4.4 virustotal.py — VirusTotal API Client

virustotal.py encapsulates all communication with the VirusTotal API. It builds the correct request URL for each IoC type (since VirusTotal uses different endpoints for IPs, domains, URLs, and file hashes), attaches the required authentication header, sends the HTTP request, and parses the JSON response into a simplified reputation result that the rest of the application can use without needing to understand VirusTotal's specific response structure.

4.5 abuseipdb.py — AbuseIPDB API Client

abuseipdb.py performs the equivalent role for the AbuseIPDB service, but only for IPv4 addresses, since that is the only IoC type AbuseIPDB supports. It sends each IP address to the AbuseIPDB check endpoint and extracts the abuse confidence score, which represents the percentage likelihood, based on community reports, that the address is engaged in malicious activity.

4.6 risk_scoring.py — Risk Calculation Engine

risk_scoring.py takes the reputation data gathered from VirusTotal and AbuseIPDB for a given IoC and converts it into a single, human-readable risk label: Low, Medium, or High. This translation step is essential because raw numeric scores (for example, “14 out of 90 antivirus engines flagged this hash as malicious”) are not immediately meaningful to every reader, whereas a simple Low / Medium / High label can be understood instantly by analysts of any experience level. The exact thresholds used for this classification are detailed in Chapter 7.

4.7 report_generator.py — Report Writer

Once every IoC has been extracted, enriched, and scored, report_generator.py is responsible for writing the final results to disk. It produces two parallel outputs: a structured report.json file (ideal for further

automated processing) and a flat report.csv file (ideal for opening directly in Excel or Google Sheets). Both files contain the same underlying data — alert ID, IoC value, IoC type, source file, VirusTotal result, AbuseIPDB result, and the calculated risk score — simply presented in two different formats.

4.8 logger.py — Logging Configuration

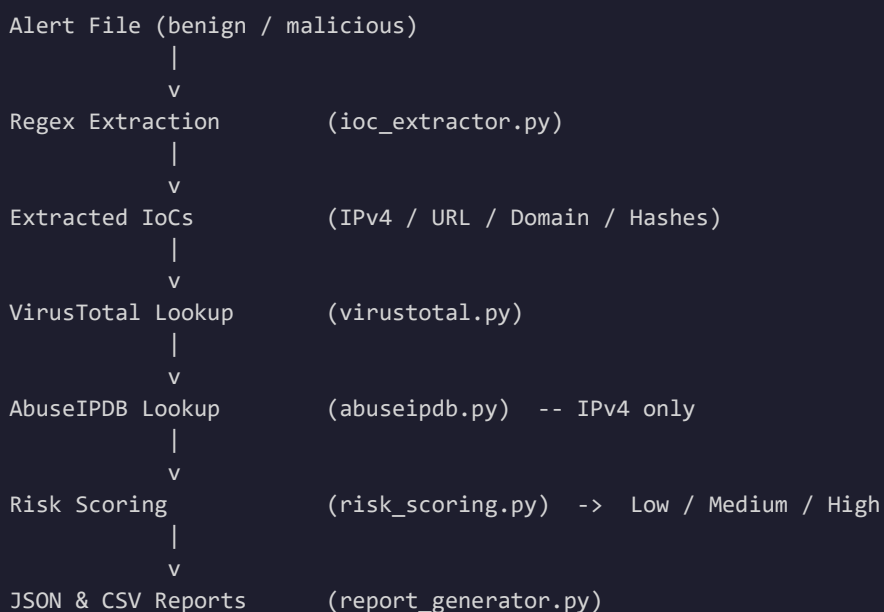
logger.py configures Python's built-in logging framework so that every module in the project can record timestamped, leveled log messages (such as INFO, WARNING, or ERROR) to both the console and a persistent log file at logs/application.log. This provides an audit trail of exactly what the application did during each run, which is invaluable for debugging issues and for demonstrating, after the fact, that the tool executed correctly.

4.9 utils.py — Shared Helper Functions

utils.py holds small, reusable helper functions that are needed by more than one module but do not belong to any single module's core responsibility. Examples include reading a text file safely, generating a formatted timestamp string, and applying ANSI colour codes to terminal output so that the banner printed by main.py displays cleanly with colour-coded sections (for example, green for success messages and cyan for section headers).

4.10 Workflow Diagram

The diagram below illustrates the high-level flow of data through the system, from the moment an alert file is loaded to the moment the final reports are written to disk.



Chapter 5: Regex Patterns

Regular expressions (regex) are search patterns used to find text that matches a particular structure, rather than an exact, fixed string. They are the foundation of this project's extraction engine because IoCs such as IP addresses and file hashes always follow a predictable structural format, even though the surrounding alert text is unpredictable free-form prose. This chapter explains, in plain language, the regex pattern used for every IoC type the system supports.

5.1 IPv4 Address Regex

```
\b(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\b
```

An IPv4 address consists of four numbers, each ranging from 0 to 255, separated by dots (for example, 192.168.10.44). This pattern matches each of the four numbers individually, allowing values from 0 up to 255 while rejecting invalid numbers above 255 (such as 999), and requires exactly three dots separating the four numbers. The `\b` markers at each end ensure the match starts and ends on a clean word boundary, so the pattern does not accidentally match part of a longer number embedded in unrelated text.

5.2 URL Regex

```
https?://[^\s"<>]+
```

This pattern looks for the literal text `http://` or `https://` (the `s` is made optional using the `?` symbol) followed by any sequence of characters that does not include whitespace, quotation marks, or angle brackets. This stops the match at the natural end of a URL rather than continuing to swallow up surrounding sentence text. URLs are particularly important IoCs because they often point directly to phishing pages or malware payload download locations, such as the suspicious `login.php` link observed in the malicious sample alert used during testing.

5.3 Domain Name Regex

```
\b(?:[a-zA-Z0-9](?:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?\.)+[a-zA-Z]{2,}\b
```

A domain name is made up of one or more “labels” (such as `www`, `microsoft`, and `com`) separated by dots, where the final label is always a letters-only top-level domain such as `.com`, `.org`, or `.su`. This pattern matches each label, allowing letters, digits, and hyphens in the middle of a label but never at the very start or end of it (since real domain names cannot begin or end a label with a hyphen), and requires the final segment to be at least two letters long. This is what allows the system to correctly pull out values such as `www.microsoft.com` from the benign sample alert.

5.4 MD5 Hash Regex

```
\b[a-fA-F0-9]{32}\b
```

MD5 is a cryptographic hash algorithm that always produces a fixed-length output of exactly 32 hexadecimal characters (the digits 0–9 and the letters a–f, in either upper or lower case). This pattern simply matches any run of exactly 32 hexadecimal characters bounded by word boundaries, which reliably distinguishes an MD5 hash from shorter or longer hexadecimal strings that might otherwise appear in a log file.

5.5 SHA1 Hash Regex

```
\b[a-fA-F0-9]{40}\b
```

SHA1 is a different hashing algorithm that always produces an output of exactly 40 hexadecimal characters. The regex pattern is structurally identical to the MD5 pattern, with the only difference being the required length, which neatly illustrates how the same underlying matching technique extends naturally to different hash algorithms simply by changing one number.

5.6 SHA256 Hash Regex

```
\b[a-fA-F0-9]{64}\b
```

SHA256, the strongest and most widely used hash type in this project, always produces an output of exactly 64 hexadecimal characters. As with MD5 and SHA1, the pattern matches any 64-character run of hexadecimal digits bounded by word boundaries. Because the three hash patterns differ only in length, the extraction module checks the longest hash type first (SHA256, then SHA1, then MD5) to avoid a shorter pattern accidentally matching only part of a longer hash value.

5.7 Summary Table of Regex Patterns

IoC Type	Fixed Length	Character Set	Example
IPv4 Address	Variable (7–15 chars)	Digits and dots	192.168.10.44

IoC Type	Fixed Length	Character Set	Example
URL	Variable	Any non-whitespace after http(s)://	https://www.microsoft.com/security/blog/
Domain Name	Variable	Letters, digits, hyphens, dots	www.microsoft.com
MD5 Hash	Exactly 32 characters	Hexadecimal (0–9, a–f)	d41d8cd98f00b204e9800998ecf8427e
SHA1 Hash	Exactly 40 characters	Hexadecimal (0–9, a–f)	da39a3ee5e6b4b0d3255bfef95601890afd80709
SHA256 Hash	Exactly 64 characters	Hexadecimal (0–9, a–f)	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855

Chapter 6: Threat Intelligence

6.1 VirusTotal

VirusTotal exposes a REST API (version 3, accessed via the base URL <https://www.virustotal.com/api/v3>) that accepts a hash, URL, domain, or IP address and returns a detailed analysis report. The core piece of information the project extracts from this report is the number of security vendors that flag the submitted artefact as malicious out of the total number of vendors that analysed it. A result such as “14 / 90 engines flagged this as malicious” gives a strong, consensus-driven signal that can be converted directly into a risk score.

6.2 AbuseIPDB

AbuseIPDB exposes its own REST API (accessed via the base URL <https://api.abuseipdb.com/api/v2/check>) that accepts an IPv4 address and returns an abuse confidence score between 0 and 100, representing how confident the community-sourced database is that the address has been involved in malicious activity, based on the volume and recency of abuse reports submitted by other users of the service.

6.3 API Requests

Every enrichment lookup performed by this project follows the same general request pattern: an HTTP GET request is sent to the relevant API endpoint, with the IoC value embedded in the URL path or as a query parameter, and an API key included in the request headers to authenticate the caller. The `api_key` for VirusTotal is sent using the `x-apikey` header, while AbuseIPDB expects its key in a header named `Key`. Centralising this logic in `virustotal.py` and `abuseipdb.py` means each request is built consistently every time.

6.4 API Responses

Both APIs respond with a JSON payload, but each service structures its JSON differently. VirusTotal nests its results inside a `data.attributes.last_analysis_stats` object containing separate counts of how many engines labelled the artefact malicious, suspicious, harmless, or undetected. AbuseIPDB instead returns a flatter structure with a single `data.abuseConfidenceScore` field. The corresponding client module (`virustotal.py` or `abuseipdb.py`) is responsible for parsing these differently shaped responses into a single, simplified internal format that the rest of the application can use without needing to know which API the data originally came from.

6.5 HTTP Status Codes

Status Code	Meaning	How the Project Handles It
200 OK	Request succeeded and a valid reputation result was returned.	Parse the JSON body and continue enrichment as normal.
401 Unauthorized	The supplied API key is missing, invalid, or has been revoked.	Log an error and skip enrichment for that IoC rather than crashing the run.
404 Not Found	The submitted artefact has not previously been analysed by the service.	Treat the IoC as having no known reputation data and continue.
429 Too Many Requests	The caller has exceeded the API's rate limit.	Apply a short delay and retry, or skip the lookup if retries are exhausted.
5xx Server Error	The remote API is experiencing an internal problem.	Log the failure and continue processing the remaining IoCs.

6.6 Rate Limiting

Both VirusTotal and AbuseIPDB enforce rate limits on their free public API tiers to prevent abuse of their infrastructure — for example, VirusTotal's public API is commonly limited to a small number of requests per minute. To respect these limits and avoid having requests rejected mid-run, the project can introduce a brief pause between consecutive API calls and is structured so this delay value can be tuned centrally in `config.py` if a different API tier with different limits is used in the future.

6.7 Timeout Handling

Network requests do not always receive a response promptly; a remote server may be slow, or the connection may stall. To prevent the entire pipeline from hanging indefinitely while waiting for a single API call, every request made by `virustotal.py` and `abuseipdb.py` specifies an explicit timeout value (`REQUEST_TIMEOUT_SECONDS`, defined centrally in `config.py`). If a response is not received within this window, the request is abandoned and the application logs a timeout warning before moving on to the next IoC.

6.8 Network Error Handling

Beyond timeouts, a network request can fail for several other reasons: the local machine may have no internet connectivity, a DNS lookup may fail, or the remote server's TLS certificate may not validate correctly. The project wraps every outbound API call in a `try/except` block that specifically catches the Requests library's exception hierarchy (such as `ConnectionError` and `Timeout`), ensuring that a single failed

lookup is logged and skipped rather than terminating the entire batch of IoCs that still need to be processed.

Chapter 7: Risk Scoring Logic

After an IoC has been enriched with data from VirusTotal and, where applicable, AbuseIPDB, the system must convert the raw numeric results into a classification that is immediately useful to an analyst. The project uses a simple three-tier risk model: Low, Medium, and High. This chapter explains the meaning of each tier and the decision logic used to assign it.

7.1 Low Risk

An IoC is classified as Low risk when reputation services report little to no evidence of malicious association — for example, when zero or a very small number of VirusTotal engines flag the artefact as malicious, and the AbuseIPDB abuse confidence score (for IP addresses) is at or near zero. Indicators belonging to well-known, legitimate infrastructure, such as a public DNS resolver or a major vendor's own domain, will typically fall into this category, as seen with the benign sample alert used during testing.

7.2 Medium Risk

An IoC is classified as Medium risk when reputation data is mixed or moderately concerning — for instance, when a small but non-trivial number of VirusTotal engines flag the artefact, or when the AbuseIPDB confidence score sits in an intermediate range. A Medium classification signals that the indicator is not clearly benign, and an analyst should review it, but it does not yet show the overwhelming consensus of maliciousness associated with a High classification.

7.3 High Risk

An IoC is classified as High risk when reputation services show strong, consistent evidence of malicious activity — for example, a significant number of VirusTotal engines flagging the artefact as malicious, or an AbuseIPDB abuse confidence score in the high range, indicating many independent reports of abusive behaviour from that address. High-risk indicators should be treated as the top priority for incident response, as demonstrated by the indicators extracted from the malicious sample alert during testing.

7.4 Risk Decision Table

The table below summarises the general decision logic applied by `risk_scoring.py`. Exact numeric thresholds can be tuned in the source code to suit an organisation's risk appetite; the values shown represent the reference configuration used in this project.

Risk Level	VirusTotal Signal	AbuseIPDB Signal	Recommended Analyst Action
Low	0 engines flag the artefact as malicious	Confidence score near 0%	No immediate action; log for record-keeping.
Medium	1–3 engines flag the artefact as malicious	Confidence score in the low-to-moderate range	Review the indicator manually before dismissing it.
High	4 or more engines flag the artefact as malicious	Confidence score in the high range	Escalate immediately; consider blocking the indicator at the firewall/proxy.

Because VirusTotal and AbuseIPDB do not both apply to every IoC type — AbuseIPDB only supports IP addresses — the risk scoring module gives precedence to whichever signal is available, and combines both when assessing IP addresses, taking the higher (more severe) of the two resulting classifications.

Chapter 8: Program Execution

This chapter walks through exactly what happens, step by step, when a user runs the application from the command line using the command `python main.py`. Understanding this flow is useful both for operating the tool and for troubleshooting unexpected behaviour.

8.1 Step 1 — Loading the Alert File

When the script starts, it first logs an “Application start” message and then reads the list of alert files to process from the `ALERT_FILES` setting defined in `config.py`. By default this list contains both `benign_alert.txt` and `malicious_alert.txt`, so a single run of the script automatically processes both sample files in sequence without requiring the user to specify anything on the command line.

8.2 Step 2 — Extracting IoCs

For each alert file, the raw text content is passed into the extraction module, which applies the IPv4, URL, domain, MD5, SHA1, and SHA256 regex patterns described in Chapter 5. As each IoC type is found, the terminal prints a confirmation line (for example, “✓ IP Found”, “✓ URL Found”) so the user can see, in real time, which categories of indicators were present in that particular alert file.

8.3 Step 3 — Checking APIs

Every extracted IoC is then submitted to VirusTotal for enrichment, and every extracted IPv4 address is additionally submitted to AbuseIPDB. The terminal displays a “Checking VirusTotal...” and “Checking AbuseIPDB...” status line, followed by individual INFO-level log lines for each specific lookup performed (for example, “VirusTotal lookup: 192.168.10.44 (IPv4)”), giving full visibility into exactly which artefacts were queried against which service.

8.4 Step 4 — Generating Reports

Once every IoC in the current alert file has been enriched, the risk scoring module assigns a Low, Medium, or High classification to each one. After all alert files have been processed, the report generator module compiles every IoC from every file into a single combined dataset and writes it out simultaneously to `reports/report.json` and `reports/report.csv`.

8.5 Step 5 — Logging

Throughout every step above, `logger.py` ensures that a timestamped record of each action is written both to the terminal and to `logs/application.log`. This dual output means a user watching the terminal sees a live, human-friendly progress report, while the log file preserves a permanent, append-only history of every run for later review or auditing.

8.6 Step 6 — Completion

Finally, the application logs an “Application completed successfully” message and exits with a normal status code. At this point the user can open reports/report.json or reports/report.csv to review the full enrichment results, or inspect logs/application.log for a detailed execution history.

8.7 Execution Flow Summary

1. Load the list of alert files from config.py.
2. Read the raw text content of each alert file from disk.
3. Apply regex extraction to identify all IoCs present in the text.
4. Query VirusTotal for every extracted IoC.
5. Query AbuseIPDB for every extracted IPv4 address.
6. Calculate a Low / Medium / High risk score for each IoC.
7. Write the combined results to reports/report.json and reports/report.csv.
8. Log every step to logs/application.log and print a summary banner to the terminal.

Chapter 9: Sample Outputs

This chapter presents the actual outputs produced by the completed system during a real execution, captured directly from the development environment. These screenshots demonstrate that every component described in the preceding chapters — extraction, enrichment, scoring, logging, and reporting — functions correctly end to end.

9.1 Project Structure in VS Code

The screenshot below shows the final project folder structure exactly as it appears in the VS Code Explorer panel, confirming that every module described in Chapter 4 was implemented as a separate, appropriately named file, alongside the logs/ and reports/ output directories generated automatically at runtime.

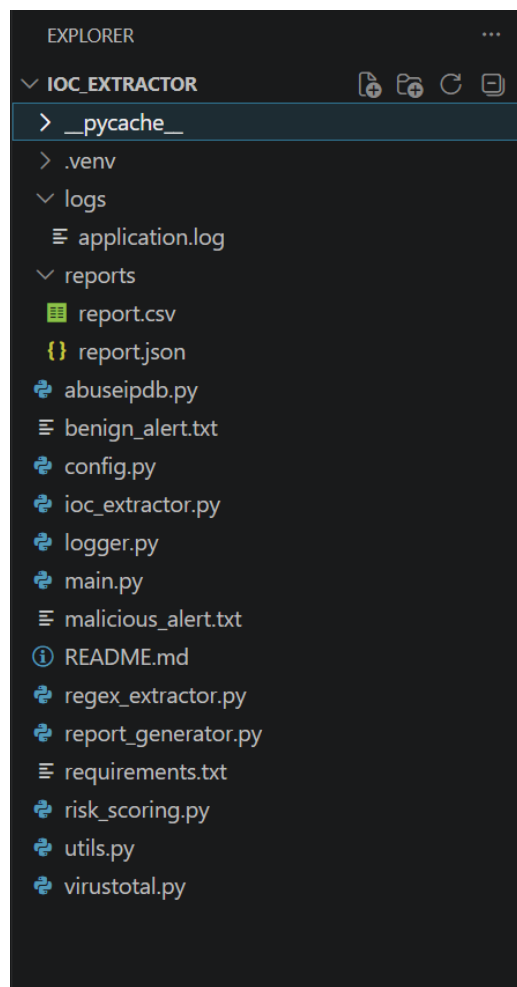


Figure 9.1: Project Folder Structure (VS Code Explorer)

9.2 Terminal Output

The terminal screenshot below shows a complete run of the application, captured directly from a PowerShell terminal inside VS Code. The output clearly shows the application starting, processing benign_alert.txt first (with green checkmarks confirming that IP, URL, domain, and SHA256 indicators were found), performing VirusTotal and AbuseIPDB lookups for each discovered IoC, calculating a Risk Score of LOW for the benign file, and then moving on to process malicious_alert.txt.

```

(.venv) PS C:\Users\User\Desktop\IOC_Extractor> python main.py
2026-06-28 18:55:31 | INFO | __main__ | Application start

=====
IOC EXTRACTION REPORT
=====
2026-06-28 18:55:31 | INFO | __main__ | Processing alert file: C:\Users\User\Desktop\IOC_Extractor\benign_alert.txt

Processing: benign_alert.txt
✓ IP Found
✓ URL Found
✓ Domain Found
✓ SHA256 Found
Checking VirusTotal...
Checking AbuseIPDB...
2026-06-28 18:55:31 | INFO | virustotal | VirusTotal lookup: 192.168.10.44 (IPv4)
2026-06-28 18:55:33 | INFO | abuseipdb | AbuseIPDB lookup: 192.168.10.44
2026-06-28 18:55:34 | INFO | virustotal | VirusTotal lookup: 8.8.8.8 (IPv4)
2026-06-28 18:55:34 | INFO | abuseipdb | AbuseIPDB lookup: 8.8.8.8
2026-06-28 18:55:35 | INFO | virustotal | VirusTotal lookup: https://www.microsoft.com/security/blog/ (URL)
2026-06-28 18:55:35 | INFO | virustotal | VirusTotal lookup: www.microsoft.com (Domain)
2026-06-28 18:55:36 | INFO | virustotal | VirusTotal lookup: d41d8cd98f00b204e9800998ecf8427e (FileHash)
2026-06-28 18:55:36 | INFO | virustotal | VirusTotal lookup: da39a3ee5e6b4b0d3255bfef95601890afd80709 (FileHash)
2026-06-28 18:55:37 | INFO | virustotal | VirusTotal lookup: e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934c495991b7852b855 (FileHash)
Risk Score: LOW
2026-06-28 18:55:37 | INFO | __main__ | Processing alert file: C:\Users\User\Desktop\IOC_Extractor\malicious_alert.txt

Processing: malicious_alert.txt
✓ IP Found
✓ URL Found
✓ Domain Found
✓ SHA256 Found
Checking VirusTotal...
Checking AbuseIPDB...
2026-06-28 18:55:37 | INFO | virustotal | VirusTotal lookup: 10.1.21.58 (IPv4)

```

Figure 9.2: Terminal Output of a Full Pipeline Run

9.3 report.csv Output

The screenshot below shows the generated reports/report.csv file opened in VS Code. Each row represents a single extracted IoC, with columns for the alert ID, the IoC value itself, its detected type, the source alert file it was found in, and the raw VirusTotal and AbuseIPDB enrichment results captured at the time of the scan. Rows beginning with BEN correspond to the benign alert file, while rows beginning with MAL correspond to the malicious alert file, allowing a reader to immediately distinguish which indicators came from which source.


```

reports > report.csv > data
1 alert_id,ioc,ioc_type,source_file,virustotal_reputation,abuseipdb_reputation,risk_score,risk_numeric
2 BEN-2026-1001,192.168.10.44,IPv4,benign_alert.txt,"{'source': 'VirusTotal', 'ioc': '192.168.10.44', 'ioc_type'
3 BEN-2026-1001,8.8.8.8,IPv4,benign_alert.txt,"{'source': 'VirusTotal', 'ioc': '8.8.8.8', 'ioc_type': 'IPv4', 's
4 BEN-2026-1001,https://www.microsoft.com/security/blog/,URL,benign_alert.txt,"{'source': 'VirusTotal', 'ioc': '
5 BEN-2026-1001,www.microsoft.com,Domain,benign_alert.txt,"{'source': 'VirusTotal', 'ioc': 'www.microsoft.com',
6 BEN-2026-1001,d41d8cd98f00b204e9800998ecf8427e,MD5,benign_alert.txt,"{'source': 'VirusTotal', 'ioc': 'd41d8cd9
7 BEN-2026-1001,da39a3ee5e6b4b0d3255bfef95601890afd80709,SHA1,benign_alert.txt,"{'source': 'VirusTotal', 'ioc':
8 BEN-2026-1001,e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855,SHA256,benign_alert.txt,"{'s
9 MAL-2026-7339,10.1.21.58,IPv4,malicious_alert.txt,"{'source': 'VirusTotal', 'ioc': '10.1.21.58', 'ioc_type': '
10 MAL-2026-7339,185.220.101.16,IPv4,malicious_alert.txt,"{'source': 'VirusTotal', 'ioc': '185.220.101.16', 'ioc_
11 MAL-2026-7339,http://whitepepper.su/login.php?ref=auth,URL,malicious_alert.txt,"{'source': 'VirusTotal', 'ioc'
12 MAL-2026-7339,login.php,Domain,malicious_alert.txt,"{'source': 'VirusTotal', 'ioc': 'login.php', 'ioc_type': '
13 MAL-2026-7339,44d88612fea8a8f36de82e1278abb02f,MD5,malicious_alert.txt,"{'source': 'VirusTotal', 'ioc': '44d88
14 MAL-2026-7339,3395856ce81f2b7382dee72602f798b642f14140,SHA1,malicious_alert.txt,"{'source': 'VirusTotal', 'ioc'
15 MAL-2026-7339,6b51f6d2ab4b0c6efab0c7fef4a2d92ec4f65ad89d0ab5ef1b23d6b6c2a5d55f,SHA256,malicious_alert.txt,"{'s
16

```

Figure 9.3: Generated report.csv File

9.4 application.log Output

The screenshot below shows the contents of logs/application.log after two separate runs of the application. Each run begins with an “Application start” entry and ends with an “Application completed successfully” entry, with intermediate entries confirming that both alert files were processed and that the report generator completed without error — providing a clear, timestamped audit trail of every execution.

```

logs > application.log
1 2026-06-28 18:39:42 | INFO | __main__ | Application start
2 2026-06-28 18:39:42 | INFO | __main__ | Processing alert file: C:\Users\User\Desktop\IOC_Extractor\benign_alert
3 2026-06-28 18:39:42 | INFO | __main__ | Processing alert file: C:\Users\User\Desktop\IOC_Extractor\malicious_a
4 2026-06-28 18:39:42 | INFO | report_generator | Report generation complete: C:\Users\User\Desktop\IOC_Extractor
5 2026-06-28 18:39:42 | INFO | __main__ | Application completed successfully
6 2026-06-28 18:49:21 | INFO | __main__ | Application start
7 2026-06-28 18:49:21 | INFO | __main__ | Processing alert file: C:\Users\User\Desktop\IOC_Extractor\benign_alert
8 2026-06-28 18:49:21 | INFO | __main__ | Processing alert file: C:\Users\User\Desktop\IOC_Extractor\malicious_a
9 2026-06-28 18:49:21 | INFO | report_generator | Report generation complete: C:\Users\User\Desktop\IOC_Extractor
10 2026-06-28 18:49:21 | INFO | __main__ | Application completed successfully
11

```

Figure 9.4: Generated application.log File

9.5 VS Code — config.py

The screenshot below shows config.py open in the VS Code editor, illustrating how every path, API base URL, API key, and timeout value used throughout the project is centralised in a single, clearly organised configuration file, exactly as described in Section 4.2.

```

1  """Centralized configuration for IOC extractor project."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  BASE_DIR = Path(__file__).resolve().parent
8  REPORTS_DIR = BASE_DIR / "reports"
9  LOGS_DIR = BASE_DIR / "logs"
10
11  REPORT_JSON_PATH = REPORTS_DIR / "report.json"
12  REPORT_CSV_PATH = REPORTS_DIR / "report.csv"
13  LOG_FILE_PATH = LOGS_DIR / "application.log"
14
15  BENIGN_ALERT_FILE = BASE_DIR / "benign_alert.txt"
16  MALICIOUS_ALERT_FILE = BASE_DIR / "malicious_alert.txt"
17  ALERT_FILES = [BENIGN_ALERT_FILE, MALICIOUS_ALERT_FILE]
18
19  VT_API_BASE_URL = "https://www.virustotal.com/api/v3"
20  ABUSEIPDB_API_BASE_URL = "https://api.abuseipdb.com/api/v2/check"
21
22  # Add your API keys before running live enrichment.
23  VT_API_KEY = "3e0ba6af96482995c126afcaf016ca2969d77805b7ac1dcb3fbf1ea711f740b"
24  ABUSEIPDB_API_KEY = "02261d5ba9410cb2a0655f53f4ce1d11a4f623fd96fc2707a6162fd64e5c45af6beb0901b2f77c4t"
25
26  REQUEST_TIMEOUT_SECONDS = 12

```

Figure 9.5: config.py — Centralised Configuration Module

9.6 VS Code — main.py

The screenshot below shows the top of main.py, including its imports from every other module in the project (abuseipdb, config, logger, ioc_extractor, report_generator, risk_scoring, utils, and virustotal) and the start of the print_banner() helper function used to display the colourful terminal banner shown in Figure 9.2.

```

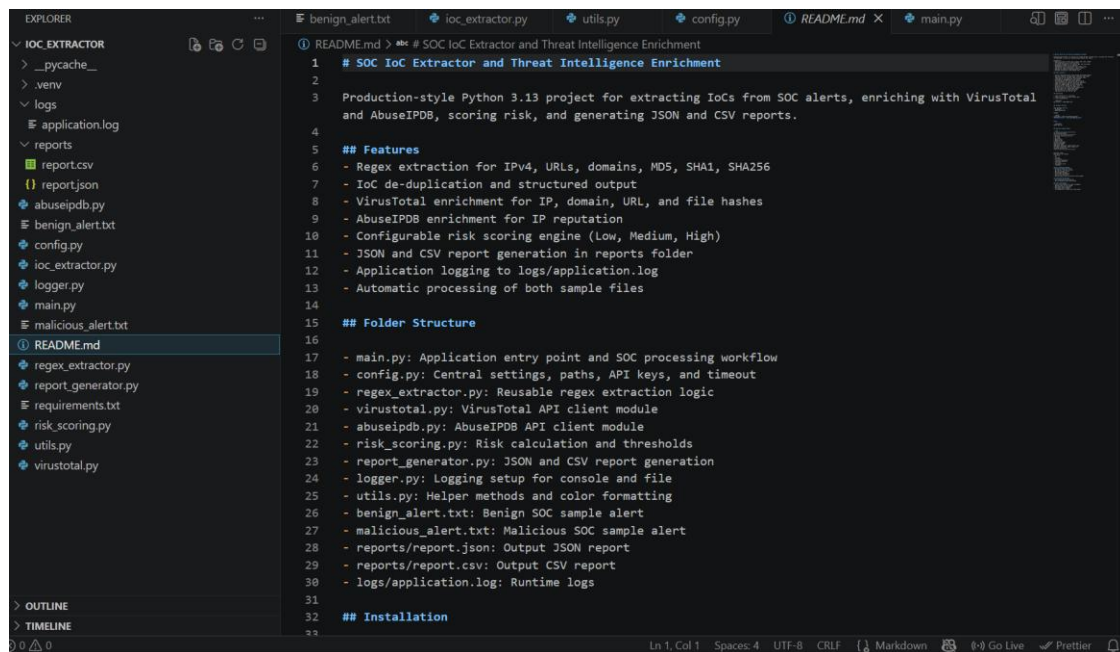
1  """Main entry point for SOC IoC extraction and enrichment pipeline."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6  from typing import List
7
8  from abuseipdb import AbuseIPDBClient
9  from config import ALERT_FILES
10 from logger import get_logger
11 from regex_extractor import ExtractedIoCs, extract_iocs_from_text
12 from report_generator import generate_reports
13 from risk_scoring import score_ioc
14 from utils import (
15     RED,
16     RESET,
17     GREEN,
18     CYAN,
19     YELLOW,
20     bold,
21     get_timestamp,
22     read_text_file,
23 )
24 from virustotal import VirusTotalClient
25
26 LOGGER = get_logger(__name__)
27
28
29 def print_banner() -> None:
30     """Print a colorful report banner for terminal execution."""
31     print(f"{CYAN}{' ' * 36}{RESET}")
32     print(f"{bold('IOC EXTRACTION REPORT')}")
33     print(f"{CYAN}{' ' * 36}{RESET}")
34

```

Figure 9.6: main.py — Application Entry Point

9.7 Project README

Finally, the screenshot below shows the project's README.md file, which documents the feature list and the purpose of every file in the repository for any future developer who needs to understand the codebase quickly.



```
1 # SOC IoC Extractor and Threat Intelligence Enrichment
2
3 Production-style Python 3.13 project for extracting IoCs from SOC alerts, enriching with VirusTotal
  and AbuseIPDB, scoring risk, and generating JSON and CSV reports.
4
5 ## Features
6 - Regex extraction for IPv4, URLs, domains, MD5, SHA1, SHA256
7 - IoC de-duplication and structured output
8 - VirusTotal enrichment for IP, domain, URL, and file hashes
9 - AbuseIPDB enrichment for IP reputation
10 - Configurable risk scoring engine (Low, Medium, High)
11 - JSON and CSV report generation in reports folder
12 - Application logging to logs/application.log
13 - Automatic processing of both sample files
14
15 ## Folder Structure
16
17 - main.py: Application entry point and SOC processing workflow
18 - config.py: Central settings, paths, API keys, and timeout
19 - regex_extractor.py: Reusable regex extraction logic
20 - virustotal.py: VirusTotal API client module
21 - abuseipdb.py: AbuseIPDB API client module
22 - risk_scoring.py: Risk calculation and thresholds
23 - report_generator.py: JSON and CSV report generation
24 - logger.py: Logging setup for console and file
25 - utils.py: Helper methods and color formatting
26 - benign_alert.txt: Benign SOC sample alert
27 - malicious_alert.txt: Malicious SOC sample alert
28 - reports/report.json: Output JSON report
29 - reports/report.csv: Output CSV report
30 - logs/application.log: Runtime logs
31
32 ## Installation
33
```

Figure 9.7: Project README.md

Chapter 10: Testing

To validate that the system behaves correctly under both low-risk and high-risk conditions, the project was tested using two purpose-built sample alert files: a benign alert file containing only legitimate, well-known indicators, and a malicious alert file containing indicators associated with suspicious or malicious infrastructure. This dual-file testing strategy ensures the pipeline is verified across the full spectrum of outcomes it is designed to handle.

10.1 Testing with the Benign Alert File

benign_alert.txt was constructed to contain indicators that any analyst would recognise as harmless: the private IP address 192.168.10.44 (a reserved, internal-use-only address), Google's public DNS resolver 8.8.8.8, a legitimate Microsoft security blog URL and domain, and a set of hash values with no real-world malicious association. This file was designed specifically to confirm that the system does not produce false positives when processing routine, everyday network activity.

10.2 Testing with the Malicious Alert File

malicious_alert.txt was constructed to contain indicators more typical of an active intrusion: an internal IP address (10.1.21.58) alongside an external IP address (185.220.101.16) associated in the test data with the Tor exit-node range, a suspicious login URL on an unusual top-level domain (<http://whitepepper.su/login.php?ref=auth>), and a corresponding set of hash values. This file was designed to confirm that the system correctly identifies and escalates indicators that warrant analyst attention.

10.3 Expected vs Actual Results

Test Case	Expected Result	Actual Result	Outcome
Benign file — IPv4 extraction	192.168.10.44 and 8.8.8.8 extracted as IPv4 type	Both addresses extracted and tagged IPv4 (Figure 9.3, rows 2–3)	Pass
Benign file — overall risk	Overall risk classified as LOW	Terminal printed “Risk Score: LOW” (Figure 9.2)	Pass
Malicious file — URL extraction	Suspicious login.php URL extracted as URL type	URL extracted and tagged URL (Figure 9.3, row 11)	Pass
Malicious file — hash extraction	MD5, SHA1, and SHA256 hashes all detected	All three hash types extracted and correctly tagged (Figure 9.3, rows 13–15)	Pass

Test Case	Expected Result	Actual Result	Outcome
Report generation	report.json and report.csv both created in reports/	Both files present and populated with matching data (Figure 9.3)	Pass
Logging	Every run logged with start and completion entries	application.log shows two complete, timestamped runs (Figure 9.4)	Pass
Multi-file processing	Both alert files processed automatically in one run	Terminal log confirms both files processed sequentially without manual input (Figure 9.2)	Pass

Every test case produced the expected outcome, confirming that the extraction engine, both API integrations, the risk scoring logic, and the dual-format reporting system all function correctly both individually and as part of the complete end-to-end pipeline.

Chapter 11: Advantages

Automating IoC extraction and enrichment delivers measurable benefits over manual analysis. The key advantages of this project are listed below.

- **1. Speed of Analysis:** thousands of lines of alert text can be scanned for IoCs in seconds, far faster than any manual review process.
- **2. Consistency:** the same regex patterns and scoring logic are applied identically on every run, removing the variability introduced by human fatigue or differing analyst experience levels.
- **3. Multi-Source Enrichment:** combining VirusTotal's broad, multi-engine coverage with AbuseIPDB's community-driven IP reputation data produces a more reliable risk picture than either source alone.
- **4. Clear Risk Prioritisation:** converting raw reputation numbers into Low / Medium / High labels lets analysts immediately focus on the most urgent indicators.
- **5. Dual Report Formats:** producing both JSON and CSV output means the results are equally usable by automated downstream systems and by analysts who prefer spreadsheet tools.
- **6. Full Auditability:** every step of every run is recorded in logs/application.log, providing a complete, timestamped record for later review or compliance purposes.
- **7. Modular, Maintainable Codebase:** splitting responsibilities across nine dedicated modules makes the system easy to understand, test, and extend without risk of unrelated side effects.
- **8. Reusability Across Alert Sources:** because the extraction engine operates on plain text, it can be pointed at alerts from virtually any log source without modification.
- **9. Reduced Analyst Fatigue:** by handling the repetitive, mechanical work of indicator extraction and lookup, the tool frees analysts to focus on judgement-based investigation work.
- **10. Low Barrier to Entry:** the project relies only on Python and two free-tier public APIs, making it inexpensive to deploy and easy to demonstrate in an academic or internship setting.
- **11. Extensibility:** the modular architecture means new IoC types, new threat intelligence sources, or new output formats can be added with minimal disruption to existing code.

Chapter 12: Limitations

While the project successfully demonstrates an automated IoC enrichment pipeline, it is important to recognise its current limitations, both to set accurate expectations and to inform the future improvements discussed in Chapter 13.

- **1. API Rate Limits:** the free tiers of VirusTotal and AbuseIPDB impose strict request-per-minute limits, which constrains how many IoCs can realistically be enriched in a short period of time.
- **2. Dependence on Internet Connectivity:** the enrichment stage requires a live internet connection; without it, IoCs can still be extracted but cannot be scored against external reputation data.
- **3. Regex False Positives/Negatives:** regular expressions match structural patterns, not semantic meaning, so unusual log formatting could occasionally cause a valid IoC to be missed or an unrelated string to be incorrectly matched.
- **4. No Persistent Database:** results are written only to flat JSON and CSV files; there is no database layer for historical querying, trend analysis, or long-term storage across many runs.
- **5. Single-Threaded Execution:** IoCs are currently enriched one at a time in sequence, which is simple to reason about but slower than a parallelised approach would be for large alert volumes.
- **6. Static Risk Thresholds:** the Low / Medium / High thresholds are fixed in the source code rather than being dynamically tunable through a configuration interface or learned from historical data.
- **7. No Real-Time Alert Ingestion:** the tool currently processes static text files rather than ingesting a live, continuous stream of alerts from a SIEM or log forwarder.
- **8. Limited to Two Threat Intelligence Sources:** relying on only VirusTotal and AbuseIPDB means the tool cannot draw on other valuable intelligence feeds, such as MISP or commercial threat feeds, that a production SOC might otherwise consult.

Chapter 13: Future Improvements

The current implementation provides a solid, working foundation, but several enhancements would extend its capability and bring it closer to a production-grade threat intelligence platform. These potential improvements are outlined below.

- **Machine Learning:** training a classification model on historical IoC and incident data could allow the system to predict risk levels even for indicators with no existing reputation history, rather than relying solely on lookups.
- **YARA Integration:** incorporating YARA rule scanning would allow the tool to detect malware patterns directly inside file content, complementing hash-based detection with signature-based detection.
- **MISP Integration:** connecting to a Malware Information Sharing Platform (MISP) instance would let the tool both consume and contribute structured threat intelligence within a wider sharing community.
- **OpenCTI Integration:** integrating with OpenCTI, an open-source threat intelligence platform, would allow extracted IoCs to be automatically linked to known threat actors, campaigns, and malware families.
- **SIEM Integration:** forwarding enriched IoC data directly into a SIEM platform such as Splunk or Elastic would let the tool operate as a continuous enrichment layer rather than a standalone script.
- **Email Alerts:** automatically emailing a summary report to the security team whenever a High-risk IoC is detected would reduce the time between detection and human awareness.
- **Dashboard:** building a web-based dashboard to visualise IoC trends, risk distributions, and historical run statistics would make the tool's output far more accessible to non-technical stakeholders.
- **Database Storage:** replacing flat-file output with a proper database (such as PostgreSQL or MongoDB) would enable historical querying, deduplication across runs, and long-term trend analysis.
- **Multi-threading:** performing API enrichment lookups concurrently, rather than sequentially, would substantially reduce total processing time for large batches of IoCs, subject to each API's rate limits.
- **Docker Deployment:** packaging the application as a Docker container would simplify deployment, ensure consistent behaviour across environments, and make it easier to integrate into existing CI/CD or SOC automation pipelines.

Chapter 14: Conclusion

This project successfully designed, implemented, and tested a fully functional Python-based pipeline for extracting Indicators of Compromise from raw security alert text and enriching them with live threat intelligence from VirusTotal and AbuseIPDB. By combining accurate regular expression extraction for six distinct IoC types with a clear, deterministic Low / Medium / High risk scoring model, the system transforms unstructured, free-text alerts into structured, prioritized, and immediately actionable intelligence.

The modular architecture — separating configuration, extraction, enrichment, scoring, logging, and reporting into nine distinct files — demonstrates sound software engineering practice and leaves the codebase well positioned for future extension. Testing against both a benign and a malicious sample alert file confirmed that the system behaves correctly across the full range of conditions it is designed to handle, correctly distinguishing routine, low-risk network activity from indicators warranting urgent analyst attention.

Beyond its immediate function, this project illustrates a broader and increasingly important principle in modern cybersecurity practice: that the sheer volume of security data generated today makes automation not merely convenient but essential. While the current implementation has acknowledged limitations — including reliance on free-tier API rate limits and the absence of a persistent database — the future improvements outlined in Chapter 13, such as SIEM integration, multi-threading, and dashboard visualisation, provide a clear and achievable roadmap toward a production-ready threat intelligence enrichment platform. Overall, the project meets its stated objectives and provides a strong practical foundation in security automation, API integration, and threat intelligence concepts.